

移动机器人复杂场景自主导航 算法与系统实现主要工作汇报

面向长距离巡航、动态车辆会车让行与台阶通行场景

汇报内容	算法框架 / 算法原理 / 核心模块 / 模块关系 / 创新点 / 验证结果
代码范围	navigation_ws/src (重点: longdist_nav 与导航适配模块)
汇报日期	2026 年 7 月

汇报人: _____ 指导教师: _____

摘要

本项目围绕移动机器人在真实园区长距离导航中的三个关键难题展开：一是常规局部规划器能够“绕开障碍”，但缺少对道路来车、会车礼让和安全恢复条件的场景理解；二是动态目标感知结果与 TEB 局部规划器的数据结构、几何表达和运动预测接口之间存在差异；三是台阶等特殊地形会同时影响点云障碍提取和局部规划参数，单独调整某一层容易造成误检、停滞或通行不稳定。针对上述问题，我在 ROS1 Navigation 基础上构建了一个由感知适配、动态局部避障、道路让行决策、地形模式切换、导航执行与仿真验证组成的分层闭环系统。

系统以 move_base 为导航执行核心，以 TEB 为主要局部轨迹优化器，并保留 NeuPAN 作为可插拔的学习型局部规划方案。在动态障碍层，设计了 TrackedObjectArray 到 TEB ObstacleArrayMsg 的适配器，统一处理目标唯一标识、多种几何形状、速度信息和可视化预测；在行为决策层，设计了道路让行状态机，利用道路边界几何、机器人到当前目标的动态行驶方向、车辆速度投影和滑动时间窗，对“真正需要让行的来车”进行筛选，随后完成巡航中断、最近可行避让点选择、台阶模式准备、靠边等待、安全清空判定和原路线恢复。同时，系统加入避让进度看门狗、候选点事件内黑名单、导航超时与 SAFE_STOP 机制，提高了不可达、感知中断和局部规划卡滞情况下的安全性。

本项目的创新重点属于“系统算法与工程方法创新”：不是重新发明 TEB 或 NeuPAN，而是将低层轨迹避障与高层道路行为决策结合，将点云阈值与规划参数进行跨层联动，并通过配置化道路模型、模块化状态机和可复现实验工具，把已有导航算法转化为能够在 A1/A2 实际路线中运行的完整方案。

关键词：移动机器人；ROS Navigation；动态障碍；TEB；道路让行；有限状态机；台阶模式；NeuPAN

工作结论概览

核心结论

形成了一套“感知数据可用、局部规划会避、场景决策会让、特殊地形能切换、异常情况下可降级”的复杂场景导航闭环。其中，道路让行状态机是主线算法，动态障碍适配与台阶模式是其关键支撑，NeuPAN 服务化集成和多动态障碍仿真构成可扩展能力。

工作层次	已完成内容	解决的核心问题
感知接口	自定义动态目标消息；点云转激光；坐标与时间规范	不同感知来源无法直接被导航模块稳定使用
动态避障	TrackedObject → TEB 动态障碍；几何/速度/预测可视化	局部规划器缺少带速度的动态目标输入
行为决策	道路区域建模、来车确认、避让、等待、恢复状态机	窄路会车不是单纯的局部绕障问题
地形适配	台阶点云 ROI 过滤与 TEB 参数动态切换、自动恢复	台阶被误判为不可通行障碍或通行参数不匹配
规划扩展	NeuPAN Python 服务端与 nav_core C++ 插件桥接	研究型算法与 ROS move_base 生命周期难以直接衔接
验证闭环	动态障碍仿真、单元测试、A1/A2 部署配置	算法难复现、异常难定位、路线迁移成本高

核心创新与核心解决方案摘要

我的核心解决方案

在 ROS Navigation 的几何导航能力之上，增加面向道路会车和特殊地形的场景决策层；同一套动态目标感知一方面驱动 TEB 连续轨迹避障，另一方面驱动道路让行状态机完成巡航中断、靠边点选择、等待清空和路线恢复，再用 /stair_mode 联动点云过滤与规划参数。由此形成“感知理解—局部规划—行为决策—地形适配—安全恢复”的闭环。

本人创新贡献	核心技术方案	解决的关键问题	代码支撑
双层动态障碍处置	TrackedObject 同时进入 TEB 动态轨迹层与 road_yield 行为决策层	普通运动目标需要连续绕行，窄路来车则需要完整礼让，单一局部规划难以兼顾	dynamic_obstacles; road_yield
目标方向自适应道路语义	根据机器人到当前巡航目标的单位向量，实时计算车辆前后位置与对向速度	固定地图方向无法支持折返、反向巡航和目标切换	road_geometry; VehicleMonitor
空间—语义—运动—时间联合确认	道路区域、车辆类别、纵向距离、速度投影和 3/5 滑动窗口共同触发	降低道路外车辆、同向车、横穿车和单帧抖动造成的误让行	DetectionWindow; VehicleMonitor
鲁棒的事件级避让策略	最近候选排序、进度看门狗、事件内黑名单、自动换点与 SAFE_STOP	固定靠边点不可达或 move_base 长时间无进展时任务容易卡死	AvoidanceStrategy; Manager
跨层台阶模式	一个状态信号同步调整前方点云 ROI 与 TEB 速度/权重，并保存、预热、恢复	仅修改感知或仅修改规划，都难同时保证台阶可通行性和运动稳定性	StairController; 点云转换

核心方案之间的联系

动态目标接口适配解决“规划器能否看懂感知数据”；目标方向与滑动窗口解决“是否真的需要让行”；有限状态机和候选切换解决“检测到来车后怎样可靠完成任务”；台阶模式解决“避让路径包含特殊地形时怎样安全通过”。这几个部分不是彼此独立的功能点，而是围绕一次完整让行事件前后衔接的核心技术链。

向老师汇报时优先强调

最核心的创新不是简单更换了局部规划器，而是把低层动态轨迹优化、高层道路行为决策和特殊地形参数切换组合成可恢复的系统算法，使机器人从“遇到障碍会躲”提升到“理解来车场景、主动靠边、确认安全后继续任务”。

目录

摘要.....	2
工作总结概览.....	2
核心创新与核心解决方案摘要.....	4
核心方案之间的联系.....	4
1 项目背景与问题定义.....	7
1.1 应用场景.....	7
1.2 常规导航栈的能力边界.....	7
1.3 本项目聚焦的四个核心问题.....	7
2 总体算法框架.....	8
2.1 分层闭环架构.....	8
2.2 模块关系与数据接口.....	9
2.3 基础算法与本人工作的边界.....	9
3 核心算法原理.....	9
3.1 基础导航与 TEB 局部轨迹优化.....	9
3.2 动态目标到 TEB 的适配算法.....	10
3.3 道路区域几何建模.....	10
3.4 基于目标方向的来车判定.....	10
3.5 滑动时间窗与安全恢复判定.....	11
3.6 避让点选择、进度看门狗与降级策略.....	11
3.7 道路让行有限状态机.....	11
3.8 台阶场景的跨层协同适配.....	12
3.9 NeuPAN 的服务化与插件化集成.....	13
4 主要模块及相互关系.....	14
4.1 road_yield: 场景级决策核心.....	14
4.2 dynamic_obstacles: 感知到规划的接口层.....	14
4.3 pointcloud_to_laserscan 与 omni_navigation: 感知/导航基础层.....	14
4.4 teb_obstacle_sim: 可复现实验工具.....	14
5 创新点与核心解决方案.....	15
5.1 最值得向老师强调的三项创新.....	15
5.2 核心解决方案的一句话概括.....	16
6 工程实现与可靠性设计.....	16
6.1 配置驱动与路线迁移.....	16
6.2 异步回调与线程安全.....	16
6.3 失效安全设计.....	16
6.4 可观测性与调试.....	17

7 测试与验证结果.....17

 7.1 单元测试结果.....17

 7.2 场景与部署验证.....18

 7.3 当前可形成的成果证据.....18

8 当前不足与下一步工作.....18

 8.1 当前边界.....18

 8.2 下一阶段建议.....19

9 总结.....19

附录 A 核心参数与代码对应.....20

附录 B 主要代码目录.....20

附录 C 参考基础.....21

说明：目录页码由 Word/LibreOffice 打开文档时可自动更新；正文已按三级标题组织。

1 项目背景与问题定义

1.1 应用场景

项目面向园区或校园中的长距离自主巡航。机器人需要在已有地图上沿预设航点完成 A1/A2 路线，同时处理建图时不存在的车辆、狭窄道路会车、道路边缘避让点、台阶或高差区域，以及感知消息短时中断等情况。这类任务的难点不在于单次“从起点到终点”，而在于长期运行过程中对路径、场景和异常连续管理。

1.2 常规导航栈的能力边界

AMCL、全局规划器、代价地图和 TEB 能够完成定位、路径搜索与局部轨迹优化，但它们主要基于几何障碍和代价函数工作。对于窄路来车，局部规划器可能反复尝试从车辆旁边穿行、等待在道路中央或发生局部最优，而不会主动理解“驶入靠边点—等待车辆通过—再恢复巡航”这一完整行为。因此，必须在局部规划之上增加场景级决策层。

1.3 本项目聚焦的四个核心问题

- (1) 如何把带语义类别、几何形状和速度的动态目标稳定转换为 TEB 可以预测的障碍物。
- (2) 如何从大量跟踪目标中识别“位于道路内、处于机器人前方、且确实需要让行”的车辆，降低误触发。
- (3) 如何将一次来车事件组织为可恢复、可重试、可安全停止的状态机，而不是若干分散条件判断。
- (4) 如何让点云障碍提取与局部规划参数根据台阶场景协同切换，并在场景结束后可靠恢复。

问题抽象

本项目把复杂导航拆成两个互补层级：低层解决“轨迹如何绕开运动目标”，高层解决“在什么场景下应中断巡航、去哪里让、何时恢复”。这种分层避免将所有行为都压给局部规划器，也避免高层状态机直接接管底层速度控制。

2 总体算法框架

2.1 分层闭环架构

① 感知与状态输入 地图/AMCL/TF、三维点云、LaserScan、TrackedObjectArray、巡航与道路配置
② 环境表征与接口适配 点云转二维激光；动态目标几何/速度适配；道路横截面构成的可通行检测带
③ 规划与场景决策 move_base 全局执行 + TEB/NeuPAN 局部规划 + road_yield 有限状态机
④ 地形与故障管理 台阶模式、动态参数重配置、候选避让点切换、超时重试、SAFE_STOP
⑤ 运动控制与反馈 /cmd_vel 输出；目标结果、机器人位姿、感知新鲜度和进度反馈闭环

数据流方面，/tracked_objects 被同时用于两个层级：一条支路经 dynamic_obstacles 适配器转换后送入 TEB，参与连续局部轨迹优化；另一条支路直接进入 road_yield 的 VehicleMonitor，用于道路语义筛选和状态机触发。/stair_mode 则从高层决策反向作用于点云过滤和 TEB 参数，构成跨层控制通道。

2.2 模块关系与数据接口

上游	接口/数据	下游	关系说明
感知系统	/tracked_objects	dynamic_obstacles	转换目标 ID、形状、姿态和速度为 TEB 动态障碍
感知系统	/tracked_objects	road_yield::VehicleMonitor	完成车辆类别、道路区域、前后方和运动方向判断
三维点云	/pseudo_cloud_base	pointcloud_to_laserscan	按高度、角度和距离投影为 /scan；台阶模式改变前方 ROI 阈值
road_yield	move_base action	NavigationClient	发送巡航点、避让点和退出点，管理成功、失败与超时
road_yield	/stair_mode + dynamic_reconfigure	点云模块 + TEB	同步切换感知阈值与局部规划参数，并保存/恢复原配置
move_base	setPlan/ computeVelocityCommands	NeuPAN 插件	通过 ROS service 调用 Python NeuPAN 后端并返回速度指令

2.3 基础算法与本人工作的边界

项目复用了 ROS Navigation 中的 move_base、AMCL、代价地图、全局规划器和 TEB，并接入外部 NeuPAN 算法。报告中不把这些成熟算法本身作为原创成果。本人的核心工作集中在：导航算法的场景化重构、动态目标接口适配、道路让行算法、台阶跨层联动、Planner 插件桥接、配置系统、异常恢复与测试部署。这种贡献边界能够更准确地体现项目价值，也使创新点具备代码和实验依据。

3 核心算法原理

3.1 基础导航与 TEB 局部轨迹优化

系统使用 move_base 统一管理全局路径、局部代价地图和速度输出。全局规划器提供从当前位置到目标点的参考路径；TEB 将局部轨迹表示为带时间间隔的离散位姿序列，在满足速度、加速度和运动学约束的同时，优化时间、障碍距离、动态障碍、路径跟随和可行性等代价。可将其思想概括为：

$$J = w_t J_{\text{time}} + w_o J_{\text{obstacle}} + w_d J_{\text{dynamic}} + w_k J_{\text{kinematics}} + w_v J_{\text{viapoint}}$$

各项权重通过 YAML 参数配置；本项目重点开启并调高动态障碍相关代价，同时保留多同伦候选以支持左右绕行。

对于移动目标，TEB 使用障碍物当前位置和速度预测未来相对位置，因此动态障碍消息是否包含稳定 ID、正确几何形状、统一坐标系和有效速度，会直接影响局部轨迹的连续性与避障提前量。

3.2 动态目标到 TEB 的适配算法

dynamic_obstacles 定义了统一的 TrackedObject 消息，包含全局唯一 UUID、Top-1 语义类别、跟踪状态、位姿与协方差、线/角速度、运动状态、朝向可用性以及包围盒、圆柱或多边形等几何描述。适配器将其转换为 costmap_converter/ObstacleArrayMsg，具体步骤如下：

- (1) 校验目标位置、四元数、尺寸和速度，过滤无效数据及低于最小速度阈值的目标。
- (2) 使用 FNV 风格哈希将 UUID 映射为稳定整数 ID，避免不同帧中障碍身份跳变。
- (3) 根据 shape_type 生成 TEB 几何：包围盒转换为旋转四边形；圆柱转换为中心点+半径；多边形将局部顶点经姿态旋转和平移到世界坐标。
- (4) 写入目标线速度和角速度；缺少角速度时显式清零，缺少可靠几何时退化为安全圆形。
- (5) 发布实体、速度箭头和常速度预测轨迹，用于 RViz 调试感知—规划接口。

$$\hat{\mathbf{p}}_i(t + \tau) = \mathbf{p}_i(t) + \mathbf{v}_i(t) \cdot \tau$$

当前动态预测采用常速度模型；目标转弯时由高频感知消息持续更新速度，使预测在每个规划周期重新校正。

该模块只向 TEB 注入具有有效运动速度的动态目标；静止障碍仍由局部代价地图处理。这种职责分离降低了同一障碍在两套通道中的重复表达，也避免把感知噪声速度误认为真实运动。

3.3 道路区域几何建模

道路检测区域不是用单个矩形近似，而是由沿道路排列的左右边界横截面组成。相邻两组左右边界形成一个四边形道路单元，内部判断通过将四边形拆分为两个三角形并使用叉积符号完成。该表示可以逼近弯道，并通过 CSV 配置直接绑定到巡航路线的有效区间。

道路中心线由每个横截面的左右端点中点连接得到，用于投影、里程和可视化；但“前方”并不固定由边界文件顺序决定，而是根据机器人当前位置到当前巡航目标实时计算。因此同一套道路边界可以支持路线反向或折返，不需要维护两套相反方向的地图数据。

3.4 基于目标方向的来车判定

设机器人位置为 $\mathbf{r}$ ，当前巡航目标为 $\mathbf{g}$ ，车辆位置为 $\mathbf{p}$ ，车辆速度为 $\mathbf{v}$ 。首先计算机器人到当前目标的单位行驶方向：

$$\mathbf{u} = (\mathbf{g} - \mathbf{r}) / \|\mathbf{g} - \mathbf{r}\|$$

车辆相对机器人的纵向距离和沿行驶方向的速度分量分别为：

$$\mathbf{s} = (\mathbf{p} - \mathbf{r})^T \mathbf{u}, \quad \mathbf{v} // = \mathbf{v}^T \mathbf{u}$$

$s > \text{ahead_margin}$ 表示车辆在前方； $s < -\text{behind_margin}$ 表示车辆在后方。可通过最大前/后检测距离限制关注范围。

一帧感知中，目标只有同时满足“类别属于车辆集合、位置位于道路四边形带内、处于机器人前方”后才进入运动方向判断。若目标被报告为静止、没有可靠速度、速度模长低于静止阈值，或 $v_{\parallel}$ 小于负的对向速度阈值，则认为该帧存在需要让行的来车。

$$z_t = 1\{\text{road}(p) \wedge \text{ahead}(s) \wedge [\text{stationary} \vee \text{no_velocity} \vee \|v\| \leq \varepsilon_s \vee v_{\parallel} \leq -\varepsilon_o]\}$$

默认静止阈值与对向速度阈值均为 0.1 m/s。

3.5 滑动时间窗与安全恢复判定

单帧检测容易受到遮挡、误检和速度抖动影响。系统维护长度 $W=5$ 的滑动窗口，只有最近 5 帧中至少 $K=3$ 帧为正才确认来车，从而实现时间维度上的多数投票。

$$\text{YieldConfirmed}_t = 1\{\sum z_k \geq K, k \in [t-W+1, t]\}, \text{默认 } W=5, K=3$$

按 5 Hz 感知频率，相当于在约 1 秒内获得多数证据后触发。

恢复条件比触发条件更严格：机器人到达避让点后启动固定最小等待计时；计时不会被后续车辆消息反复重置。只有等待时间已满足，并且收到 5 条新的、连续的、时间新鲜的感知消息，每条消息均不存在前方来车且不存在后方配置车辆，才允许退出避让。如果感知超时，连续清空计数立即归零，避免使用旧消息恢复导航。

3.6 避让点选择、进度看门狗与降级策略

一次让行事件开始时，系统先根据当前巡航索引筛除不适用于该道路区间的避让点，再按机器人到候选点的欧氏距离排序，只保留最近的 K 个候选。

$$q^* = \arg \min d(r, q_i), q_i \in \text{Eligible}(\text{route_index}) \text{ 且 } q_i \notin \text{Blacklist}$$

默认最多保留 5 个候选，保证选择局部最合理的靠边点，同时限制搜索规模。

为识别“move_base 仍处于 ACTIVE，但机器人实际上已卡住”的情况，设计了距离进度看门狗。若在 $T=8$ s 的时间窗口内，机器人到当前避让点的距离改善不足 $\delta=0.25$ m，则取消当前目标、将该避让点加入本次事件黑名单，并立即切换到下一个最近候选。定位暂时缺失时看门狗暂停计时，避免把 TF 故障误判为路径阻塞。

$$\text{若 } d_{\text{ref}} - d(t) \geq \delta, \text{ 则更新参考距离与计时; 否则 } t - t_{\text{progress}} \geq T \text{ 时判定无进展}$$

所有候选均失败时进入 SAFE_STOP，而不是继续输出不确定动作。

3.7 道路让行有限状态机

road_yield_manager 将任务拆分为 10 个显式状态，使触发、执行、等待、恢复和故障逻辑可追踪。状态机以固定控制频率运行，感知回调只维护线程安全快照，导航 action 回调只更新结果，从而减少异步回调之间的耦合。

状态	进入条件/职责	关键输出或下一状态
INIT	等待 move_base、TF；按配置决定是否等待新鲜感知	进入 ROUTE_NAV
ROUTE_NAV	发送/跟踪当前巡航点；检查来车确认	正常前进或 PREPARE_YIELD
PREPARE_ROUTE_STAIR	巡航点被标记为台阶，先切换地形模式并预热	回到 ROUTE_NAV
PREPARE_YIELD	取消巡航目标，选择候选点，开启台阶/靠边模式	GO_TO_YIELD
GO_TO_YIELD	导航到避让点；监控 action 结果与距离进展	WAIT_CLEAR / 换点 / SAFE_STOP
WAIT_CLEAR	固定等待 + 连续新鲜清空帧	EXIT_YIELD 或 RESUME_ROUTE
EXIT_YIELD	存在退出点时驶回正常道路区域	RESUME_ROUTE
RESUME_ROUTE	清理事件黑名单和计时器，恢复中断航点	ROUTE_NAV
FINISHED	全部巡航点处理完成	终止任务
SAFE_STOP	不可恢复异常或候选耗尽	保持停止，等待人工处理

状态机的关键设计

让行不是“检测到车就发一个新目标”，而是一个带前置准备、到达确认、时间约束、感知清空、退出动作、路线恢复和失败降级的事务。每次让行事件拥有独立候选集与黑名单，事件结束后统一清理。

3.8 台阶场景的跨层协同适配

台阶问题同时发生在感知层和规划层。若点云转激光保持普通地面阈值，机器人前方台阶面可能被投影成近距离障碍，导致局部规划器认为道路不可通行；若仅过滤台阶点云而不限速度 和转向，又可能造成通行过程过激。因此本项目通过 /stair_mode 同时联动两个模块。

联动对象	普通模式	台阶模式	安全机制
点云转激光	使用全局 min_height	仅在前方 ROI 内把最小高度提高到 0.4 m	保留侧向障碍；0.5 s 消息 超时自动失效
TEB 参数	使用原始动态配置	降低线/角速度和加速度，提高前 向约束与路径跟随权重	切换前保存原参数，退出后 逐项恢复
任务管理	直接发送普通航点	先发布模式、等待 warmup，再发 送台阶/避让目标	失败、结束和 shutdown 均执行恢复

前方 ROI 的做法比全局提高点云高度阈值更稳健：它只忽略机器人正前方可能属于台阶结构的低矮点，而左右两侧和远处的真实障碍仍会进入激光扫描。/stair_mode 以 10 Hz 持续发布，点云端带超时保护，避免管理节点异常退出后长期停留在特殊过滤模式。

3.9 NeuPAN 的服务化与插件化集成

NeuPAN 运行于独立 Python/Conda 环境，而 move_base 的局部规划器接口是 C++ nav_core 插件。直接把两者写在一个进程中会带来 Python 依赖、环境冲突和生命周期管理问题。本项目将 NeuPAN 拆成持久 ROS 服务端和轻量 C++ 插件：

- (1) C++ 插件的 setPlan() 把 move_base 全局路径统一变换到 path_frame，并调用 set_path 服务。
- (2) computeVelocityCommands() 获取机器人位姿，调用 compute_velocity 服务，返回线速度、角速度、到达/停止状态及局部轨迹。
- (3) Python 服务端把 LaserScan 转为二维障碍点并变换到地图坐标，按滚动时域调用 NeuPAN，发布优化轨迹与参考轨迹。
- (4) 在 move_base 模式下关闭 Python 端直接发布 /cmd_vel，保证只有 move_base 拥有速度输出权，避免指令竞争。

该设计的价值在于把研究算法包装成标准局部规划器，使 TEB、DWA 和 NeuPAN 可以通过 launch 参数切换，同时将 Python 推理环境与 ROS 导航主进程隔离，便于迁移和故障定位。

4 主要模块及相互关系

4.1 road_yield：场景级决策核心

子模块	职责	与其他模块的联系
config_loader	读取 YAML 与巡航/避让/道路边界 CSV；完成格式校验和路径解析	向所有状态机组件提供统一 ManagerConfig
road_geometry	道路四边形包含判断、中心线投影、方向相关的前方判断	被 VehicleMonitor 和避让策略调用
pose_provider	封装 TF 位姿、点和速度向量变换	隔离坐标系细节，服务感知与进度监控
vehicle_monitor	过滤车辆、判断前后方、运动方向、滑动窗口与感知新鲜度	向 Manager 提供线程安全 PerceptionSnapshot
navigation_client	封装 move_base action、超时、取消、结果序列同步	执行巡航点、避让点和退出点
stair_controller	发布 /stair_mode，保存/修改/恢复 TEB 参数	联动点云与局部规划器
road_yield_manager	10 状态主状态机、候选切换、重试和 SAFE_STOP	协调全部子模块

4.2 dynamic_obstacles：感知到规划的接口层

该包既定义动态目标消息规范，也实现 TEB 障碍适配。消息规范明确了数组级时间戳/坐标系、目标底面中心、局部形状坐标系、速度的自运动补偿含义、跟踪状态和朝向有效性。这种接口约束减少了感知、规划双方对同一字段含义的不同理解，是系统稳定性的基础。

4.3 pointcloud_to_laserscan 与 omni_navigation：感知/导航基础层

omni_navigation 统一组织地图、AMCL、点云转激光、move_base、TEB/DWA/NeuPAN 选择和机器人型号参数。在标准 pointcloud_to_laserscan 基础上加入台阶模式订阅、前方 ROI 和超时保护，使点云转换不再是固定阈值的静态预处理，而成为可由任务场景动态控制的感知模块。

4.4 teb_obstacle_sim：可复现实验工具

仿真器支持 box、circle/cylinder/sphere、polygon 和 line 等形状，支持 square、circle、waypoints 与 stationary 路线，能够配置速度、相位、顺逆时针和朝向模式。它同时发布 TEB 动态障碍、实体标记、完整路线、速度箭头和短期预测，用于在真实感知接入前验证动态避障参数及可视化链路。

5 创新点与核心解决方案

创新定位

本项目的创新不是单点算法替换，而是把“动态障碍的连续轨迹优化”和“道路场景的离散行为决策”组合成双层体系，再通过可切换地形模式和可靠性机制将其落地到真实路线。以下创新均能在代码模块、配置和测试中找到直接对应。

创新点	传统做法的不足	本项目核心方案	实际价值
1. 双层动态障碍处置	只依赖局部规划器，窄路会车容易等待在路中或反复试探	同一感知数据同时进入 TEB 动态轨迹层和 road_yield 行为决策层	既能连续绕行普通运动目标，又能对来车执行完整礼让行为
2. 目标方向自适应道路语义	固定道路正方向在折返或反向巡航时失效	以机器人→当前目标的单位向量实时定义前方、后方和对向速度	一套边界支持双向路线和目标切换，配置复用性更高
3. 空间—语义—运动—时间联合触发	单帧或仅按距离触发误报率高	道路区域、车辆类别、纵向位置、速度投影、5 帧滑窗共同确认	过滤道路外车辆、同向车、横穿车和瞬时抖动
4. 事件级多候选避让策略	固定避让点不可达时任务容易卡死	最近 K 候选 + 进度看门狗 + 本事件黑名单 + 全失败 SAFE_STOP	面对局部阻塞能够自动换点，同时保留明确安全边界
5. 跨层可切换台阶模式	仅改点云或仅改规划参数都难兼顾可通行性和安全性	高层状态机同步控制前方 ROI 点云阈值和 TEB 动态参数，并自动恢复	减少台阶误障碍与高速不稳定，避免特殊参数污染普通路段
6. 异构规划器服务桥	Python 学习算法难直接满足 nav_core 插件与环境要求	C++ 插件负责 move_base 契约，Python 服务端负责 NeuPAN 推理	实现可插拔、可迁移、单一速度发布者和环境隔离
7. 配置—仿真—测试闭环	路线硬编码、异常难复现	三类 CSV + YAML 参数化，多形状/多路线动态障碍仿真，纯算法单测	降低路线迁移成本，提高验证可重复性和维护效率

5.1 最值得向老师强调的三项创新

- 第一， 从“避障”提升到“行为决策”：道路让行状态机把来车识别、靠边点选择、等待、清空确认和原路恢复组成完整闭环，解决了传统局部规划器无法表达交通礼让规则的问题。
- 第二， 使用目标方向而不是固定地图方向建立道路语义：位置投影和速度投影都随当前导航目标更新，使算法天然支持折返路线和反向通过同一道路。
- 第三， 跨越感知层与规划层的台阶模式：同一个状态信号同步改变点云前方 ROI 和 TEB 参数，并带超时、预热和恢复机制，体现了系统级协同设计。

5.2 核心解决方案的一句话概括

汇报表述建议

我的核心工作，是在 ROS Navigation 的连续轨迹规划之上增加一层面向道路会车和特殊地形的场景决策，并打通感知目标、局部规划器、任务状态机和参数切换之间的数据闭环，使机器人从“能导航”提升到“能在复杂规则场景下可靠完成任务”。

6 工程实现与可靠性设计

6.1 配置驱动与路线迁移

巡航点、避让点和道路左右边界分别存储在 CSV 中，行为参数统一放在 YAML。配置加载器支持相对路径解析、UTF-8 BOM、带引号 CSV、角度制转换、必需列校验、数值有限性校验、道路文件成对可用性检查和台阶航点双重标记方式。A1 可在没有道路让行文件时以纯巡航+台阶模式运行；A2 同时加载道路边界和避让点，启用来车让行。

6.2 异步回调与线程安全

VehicleMonitor 使用互斥锁维护感知快照、序号和当前行驶目标版本。当导航目标变化时清空滑动窗口和旧快照；若感知回调处理期间目标版本发生变化，则丢弃该次结果，防止旧方向的判断污染新航点。NavigationClient 通过递增序列号屏蔽已取消目标的迟到回调，避免 action 结果串扰。

6.3 失效安全设计

异常	检测方式	系统反应
感知消息过期	当前时间-接收时间 > perception_timeout	巡航阶段警告并不触发让行；等待阶段清空连续安全计数
TF/机器人位姿暂不可用	PoseProvider 变换失败	节流告警；不生成错误判断；进度看门狗暂停
巡航目标失败/超时	move_base action 状态或目标超时	按次数与距离策略重试、跳过或 SAFE_STOP
避让目标无进展	8 s 内距离改善不足 0.25 m	黑名单当前点并切换下一个候选
所有候选失败	候选集均进入事件黑名单	取消目标并进入 SAFE_STOP
台阶模式节点异常	/stair_mode 消息超时	点云端自动退出特殊过滤；管理器 shutdown 时恢复 TEB

6.4 可观测性与调试

系统为状态切换、目标发送、感知变化、候选切换、等待计时、配置加载和异常恢复提供明确日志；动态障碍适配器和仿真器在 RViz 中显示实体、速度箭头、预测轨迹和完整运动路线；NeuPAN 发布优化轨迹、参考轨迹和初始路径。这些可视化与日志使问题能够定位到感知、坐标变换、决策、规划或执行中的具体层级。

7 测试与验证结果

7.1 单元测试结果

本次基于当前工作区重新执行 road_yield 单元测试。核心算法相关测试全部通过；配置加载测试中有一项仿真样例数量断言未同步：当前 path_sim/yield_points.csv 含 9 个避让点，测试仍期望 7 个，因此出现 1 项失败。该问题属于测试基线维护问题，不涉及道路几何、来车判定或候选切换算法错误。

测试组	测试数	通过	覆盖内容	结果
RoadGeometry	4	4	空几何安全、区域内/前方、反向路线、弯道路段	通过
DetectionWindow / Oncoming	6	6	3/5 多数窗、窗口滑动、同向/横向/静止/无速度判定	通过
AvoidanceStrategy	4	4	进度超时、进度重置、位姿缺失暂停、候选排序与限制	通过
ConfigLoader	6	5	三 CSV、台阶标记、纯巡航、缺失/不完整配置、仿真配置	1 项基线未同步
合计	20	19	核心算法断言全部通过	19/20

7.2 场景与部署验证

场景	验证内容	仓库中的结果依据
A1 路线	预设航点连续导航；指定航点进入台阶模式；结束后恢复普通参数	部署说明与提交记录标注 A1 测试成功
A2 路线	道路来车检测；中断巡航；驶向靠边点；安全后恢复原路线	部署说明与提交记录标注 A2/道路让行测试成功
动态障碍仿真	多形状、多路线、速度与预测可视化；直接向 TEB 发布	teb_obstacle_sim 配置与 launch 可复现
Planner 切换	TEB、DWA、NeuPAN 通过 launch 参数选择	omni_navigation 的 move_base_* 启动文件

7.3 当前可形成的成果证据

- 形成 4 个面向复杂导航的独立功能包：dynamic_obstacles、road_yield、neupan_service、teb_obstacle_sim。
- road_yield 已由单体节点重构为配置、几何、TF、感知、导航、地形和状态机等解耦模块。
- 重点模块及定制代码约 4,870 行（不含第三方导航栈、消息注释和配置文件）。
- 已形成 A1/A2 路线部署配置、启动说明、仿真配置和 20 项单元测试。

8 当前不足与下一步工作

8.1 当前边界

- (1) 动态障碍预测目前主要采用常速度模型，尚未利用加速度、协方差和多模态轨迹预测；车辆急转弯时依赖高频更新校正。
- (2) 道路让行依赖预先采集的道路横截面和避让点，适合固定园区路线，但对完全未知道路的在线生成能力有限。
- (3) 当前感知发布端未稳定提供分类概率，因此来车筛选暂未使用置信度；后续可加入目标存在概率和类别概率门限。
- (4) 单元测试主要覆盖纯算法与配置加载，尚需增加带 mock action/TF/感知的状态机集成测试和 rosbag 回放测试。
- (5) 当前验证以功能成功为主，后续应补充成功率、最小安全距离、让行耗时、误触发率、恢复时间和计算开销等量化指标。

8.2 下一阶段建议

优先级	工作项	预期提升
P0	同步仿真配置测试断言，建立 CI 自动运行 20 项测试	消除测试基线漂移，保证每次修改可回归
P0	录制 A1/A2 rosbag，构建来车、感知中断、候选不可达回放集	将实车偶现问题转化为可重复测试
P1	增加状态机集成测试和故障注入	验证 action 迟到回调、TF 丢失、感知超时等组合情况
P1	利用协方差和加速度进行不确定性膨胀/轨迹预测	提高高速或转弯车辆的动态避障提前量
P2	在线生成道路走廊与候选靠边点	降低对人工 CSV 采集的依赖，扩展到未知路线
P2	建立量化指标与对比实验（TEB-only vs 双层方案）	更有力地证明创新方案在成功率和安全性上的收益

9 总结

本项目完成了从通用 ROS 导航栈到复杂园区任务系统的关键跨越。底层通过动态障碍适配和 TEB 参数配置获得对运动目标的连续轨迹规划能力；中层通过 move_base/Planner 接口统一管理 TEB、DWA 和 NeuPAN；高层通过道路让行状态机表达来车触发、靠边等待、清空恢复和异常降级；台阶模式则把任务语义反向作用到点云过滤和局部规划参数，实现跨层协同。

从工作贡献看，最核心的成果是 road_yield 场景决策算法及其模块化重构；最突出的创新是双层动态障碍处置、目标方向自适应的道路语义和跨层台阶模式；最重要的工程价值是将配置、仿真、测试、部署和故障恢复纳入同一闭环。这套方案不仅能完成当前 A1/A2 路线，也为后续接入更强的预测模型和学习型规划器提供了清晰接口。

最终汇报结论

我完成的不是若干导航脚本的简单拼接，而是一套面向动态会车和特殊地形的分层导航算法框架：它以成熟导航算法为基础，通过我设计的感知适配、行为状态机、跨层参数联动和故障恢复机制，使机器人在真实复杂路线中具备可解释、可配置、可恢复的自主任务能力。

附录 A 核心参数与代码对应

算法/机制	默认关键参数	主要代码位置
来车多数确认	window=5, required=3	road_yield/include/road_yield/ detection_window.h
安全清空	连续 5 条新鲜消息	road_yield/src/road_yield_manager.cpp
前/后方判断	ahead/behind margin=0.3 m, 最大 30 m	road_yield/src/vehicle_monitor.cpp
避让候选	最多 5 个; 按距离排序	road_yield/include/road_yield/ avoidance_strategy.h
进度看门狗	8 s 内至少改善 0.25 m	road_yield/include/road_yield/ avoidance_strategy.h
台阶点云 ROI	x=0.05~3.0 m, $ y \leq 1.0$ m, min z=0.4 m	pointcloud_to_laserscan_nodelet.cpp
台阶 TEB	速度、加速度与权重动态覆盖	road_yield/src/stair_controller.cpp
动态障碍预测	常速度; 预测时域/采样数参数化	dynamic_obstacles/src/ teb_obstacle_adapter.cpp
NeuPAN	receding=4, step_time=0.3 s	neupan_service/config/limo.yaml

附录 B 主要代码目录

目录	说明
src/longdist_nav/road_yield	道路巡航、来车判断、靠边让行、台阶模式和异常恢复核心
src/longdist_nav/dynamic_obstacles	动态目标消息定义与 TEB 障碍适配器
src/longdist_nav/neupan_service	NeuPAN Python 服务端、ROS service 与 nav_core 插件
src/longdist_nav/teb_obstacle_sim	多动态障碍仿真、运动路线与预测可视化
src/nav1/omni_navigation	地图、定位、move_base、Planner 选择与参数组织
src/nav1/pointcloud_to_laserscan	点云转激光及台阶前方 ROI 动态过滤

附录 C 参考基础

- [1] C. Rösmann, F. Hoffmann, T. Bertram. Integrated online trajectory planning and optimization in distinctive topologies. Robotics and Autonomous Systems, 2017.
- [2] ROS Navigation / move_base / costmap_2d / AMCL 软件框架与接口。
- [3] NeuPAN 算法及其 Python 运行库；本项目完成 ROS1 服务化与 nav_core 插件化集成。